The `PostDetails` `@OneToOne` mapping is changed as follows:

```
@OneToOne
@MapsId
private Post post;
```

This time, the table relationship does not feature any additional foreign key column since the `post_details` table primary key references the `post` table primary key:

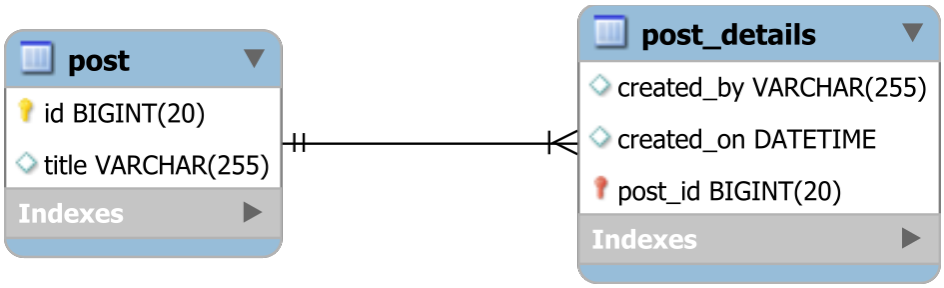

Figure 10.11: The shared key one-to-one

Because `PostDetails` has the same identifier as the parent `Post` entity, it can be fetched without having to write a JPQL query.

```
PostDetails details = entityManager.find(PostDetails.class, post.getId());
```

The shared primary key efficiency

First of all, the shared primary key approach reduces the memory footprint of the child-side table indexes since it requires a single indexed column instead of two. The more records a child table has, the better the improvement gain for reducing the number of indexed columns.

More, the child entity can now be simply retrieved from the second-level cache, therefore preventing a database hit. In the previous example, because the child entity identifier was not known, a query was inevitable. To optimize the previous use case, the *query cache* would be required as well, but the query cache is not without issues either.

Because of the reduced memory footprint and enabling the second-level cache direct retrieval, the JPA 2.0 *derived identifier* is the preferred `@OneToOne` mapping strategy. The shared primary key is not limited to unidirectional associations, being available for bidirectional `@OneToOne` relationships as well.

10.5.2 Bidirectional @OneToOne

A bidirectional `@OneToOne` association allows the parent entity to map the child-side as well:

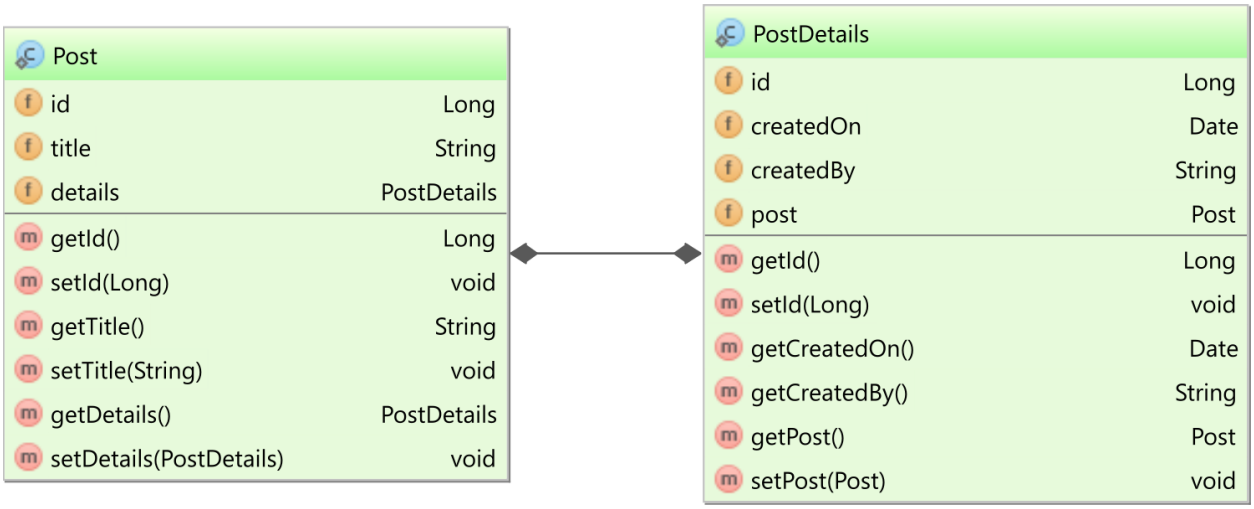

Figure 10.12: The bidirectional @OneToOne relationship

The parent-side defines a `mappedBy` attribute because the child-side (which can still share the primary key with its parent) is still in charge of this JPA relationship:

```
@OneToOne(mappedBy = "post", cascade = CascadeType.ALL, fetch = FetchType.LAZY)
private PostDetails details;
```

Because this is a bidirectional relationship, the `Post` entity must ensure that both sides of this relationship are set upon associating a `PostDetails` entity:

```
public void setDetails(PostDetails details) {
    if (details == null) {
        if (this.details != null) this.details.setPost(null);
    }
    else details.setPost(this);
    this.details = details;
}
```

Unlike the parent-side `@OneToMany` relationship where Hibernate can simply assign a proxy even if the child collection is empty, the `@OneToOne` relationship must decide if to assign the child reference to null or to an `Object`, be it the actual entity object type or a runtime Proxy.

This is an issue that affects the parent-side `@OneToOne` association, while the child-side, which has an associated foreign key column, knows whether the parent reference should be null or not. For this reason, the parent-side must execute a secondary query to know if there is a mirroring foreign key reference on the child-side.

Even if the association is lazy, when fetching a `Post` entity:

```
Post post = entityManager.find(Post.class, 1L);
```

Hibernate fetches the child entity as well, so, instead of only one query, Hibernate requires two select statements:

```
SELECT p.id AS id1_0_0_, p.title AS title2_0_0_
FROM   post p
WHERE  p.id = 1

SELECT pd.post_id AS post_id3_1_0_, pd.created_by AS created_1_1_0_,
       pd.created_on AS created_2_1_0_
FROM   post_details pd
WHERE  pd.post_id = 1
```

If the application developer only needs parent entities, the additional child-side secondary queries will be executed unnecessarily, and this might affect application performance. The more parent entities are needed to be retrieved, the more obvious the secondary queries performance impact gets.

Limitations

Even if the foreign key is `NOT NULL` and the parent-side is aware about its non-nullability through the `optional` attribute (e.g. `@OneToOne(mappedBy = "post", fetch = FetchType.LAZY, optional = false)`), Hibernate still generates a secondary select statement.

For every managed entity, the Persistence Context requires both the entity type and the identifier, so the child identifier must be known when loading the parent entity, and the only way to find the associated `post_details` primary key is to execute a secondary query. Because the child identifier is known when using `@MapsId`, in future, HHH-10771[a] should address the secondary query issue.

Bytecode enhancement is the only viable workaround. However, it only works if the parent side is annotated with `@LazyToOne(LazyToOneOption.NO_PROXY)` and the child side is not using `@MapsId`. Because it's simpler and more predictable, the unidirectional `@OneToOne` relationship is often preferred.

[a] https://hibernate.atlassian.net/browse/HHH-10771

10.6 @ManyToMany

From a database perspective, the `@ManyToMany` annotation mirrors a many-to-many table relationship:

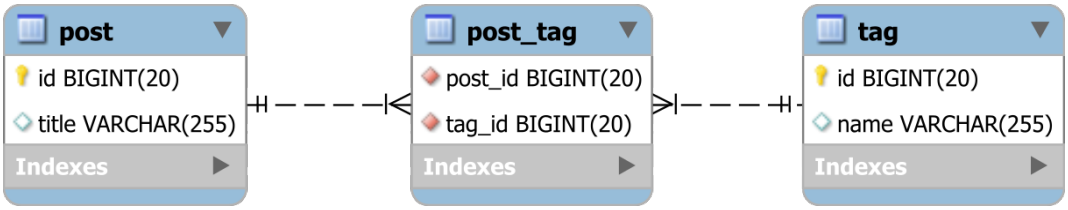

Figure 10.13: The many-to-many table relationship

Like the `@OneToMany` and `@OneToOne` relationships, the `@ManyToMany` association can be either unidirectional or bidirectional.

10.6.1 Unidirectional @ManyToMany List

In our case, if we need to map a unidirectional `@ManyToMany` association between the `Post` and `Tag` entities, it makes sense to have the `Post` entity map the `@ManyToMany` relationship since there is not much need for navigating this association from the `Tag` side (although we can still do it with a JPQL query).

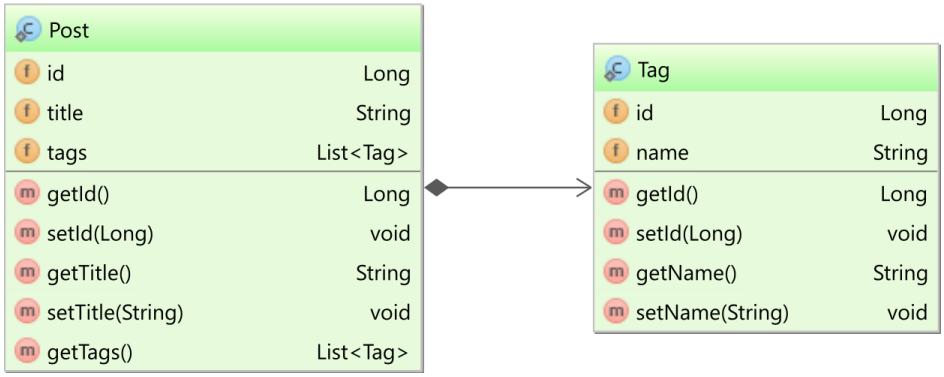

Figure 10.14: The unidirectional @ManyToMany relationship

In the `Post` entity, the `tags` collection using a `List`, like this:

```
@ManyToMany(cascade = {CascadeType.PERSIST, CascadeType.MERGE})
@JoinTable(name = "post_tag",
    joinColumns = @JoinColumn(name = "post_id"),
    inverseJoinColumns = @JoinColumn(name = "tag_id")
)
private List<Tag> tags = new ArrayList<>();
```

When adding several entities:

```
Post post1 = new Post("JPA with Hibernate");
Post post2 = new Post("Native Hibernate");

Tag tag1 = new Tag("Java");
Tag tag2 = new Tag("Hibernate");

post1.getTags().add(tag1);
post1.getTags().add(tag2);
post2.getTags().add(tag1);

entityManager.persist(post1);
entityManager.persist(post2);
```

Hibernate manages to persist both the `Post` and the `Tag` entities along with their junction records.

```
INSERT INTO post (title, id) VALUES ('JPA with Hibernate', 1)
INSERT INTO post (title, id) VALUES ('Native Hibernate', 4)

INSERT INTO tag (name, id) VALUES ('Java', 2)
INSERT INTO tag (name, id) VALUES ('Hibernate', 3)

INSERT INTO post_tag (post_id, tag_id) VALUES (1, 2)
INSERT INTO post_tag (post_id, tag_id) VALUES (1, 3)
INSERT INTO post_tag (post_id, tag_id) VALUES (4, 2)
```

Cascading

For `@ManyToMany` associations, `CascadeType.REMOVE` does not make too much sense when both sides represent independent entities. In this case, removing a `Post` entity should not trigger a `Tag` removal because the `Tag` can be referenced by other *posts* as well. The same arguments apply to orphan removal since removing an entry from the `tags` collection should only delete the junction record and not the target `Tag` entity.

For both unidirectional and bidirectional associations, it is better to avoid the `CascadeType.REMOVE` mapping. Instead of `CascadeType.ALL`, the cascade attributes should be declared explicitly (e.g. `CascadeType.PERSIST`, `CascadeType.MERGE`).